

Strategy Design



Design Pattern

→ While developing application / software there are common problems faced by every developer. Design Pattern is the standard solution to those common recurring problems.

There are total 23 design Patterns, every design Pattern High level pe ek hi baat bolta hai.

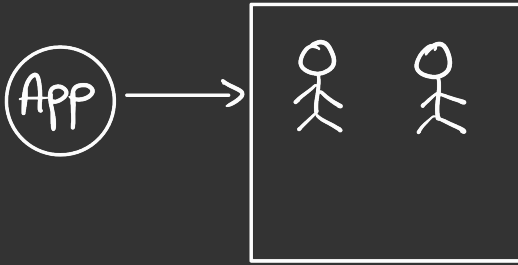
Apke application me kuch hissa aisa hoga static hoga jo ki change nhi hoga aur kuch hissa dynamic hoga jo change hota rahega.

Us dynamic hisse ko static se alag rakho.

Strategy Design Pattern

eg use case :→

EK application create kar rahe hai . Us application me hume robots ko simulate karna hai .



→ Step 1 is to create a Robot class.



→ step 2, Ask question → which features to include in robot class?

→ walk()
→ talk()
→ projection()

} → these will be common methods

robot dikhta Kaisa hai!

→ this will be abstract as har robot alag dikhega

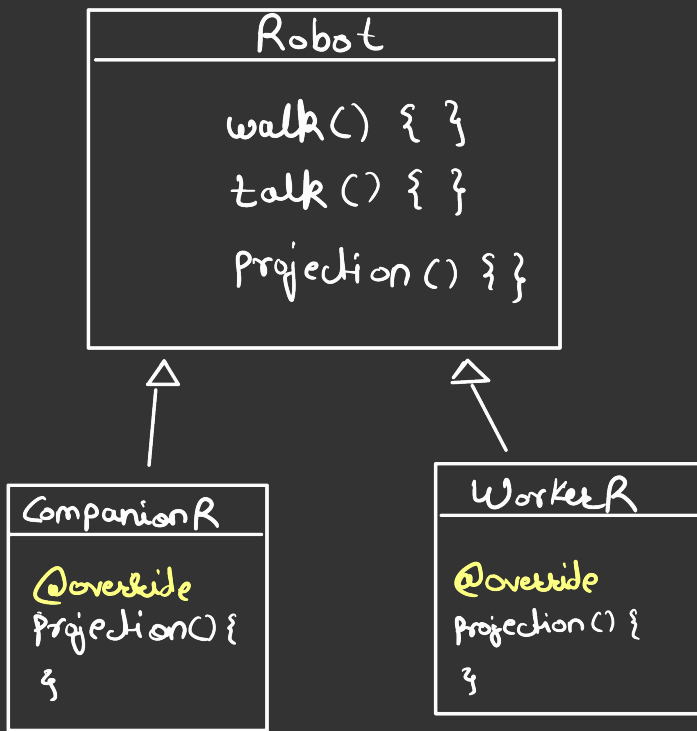
Robot
walk() { }
talk() { }
projection() { }

Note → Strategy Pattern concludes that inheritance bhot bekar hai

→ Let's say abhi humare pass bas 2 type ke Robots hai

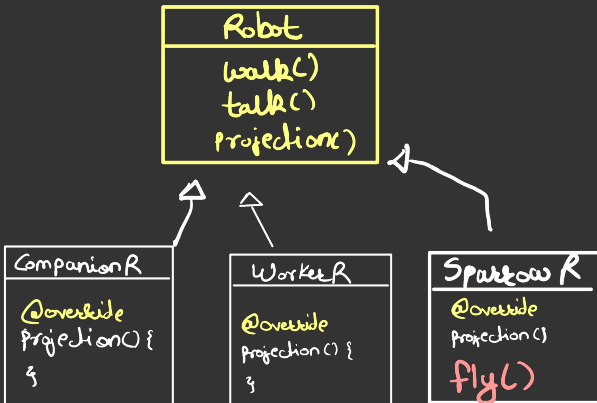
① Companion Robot

② Worker Robot

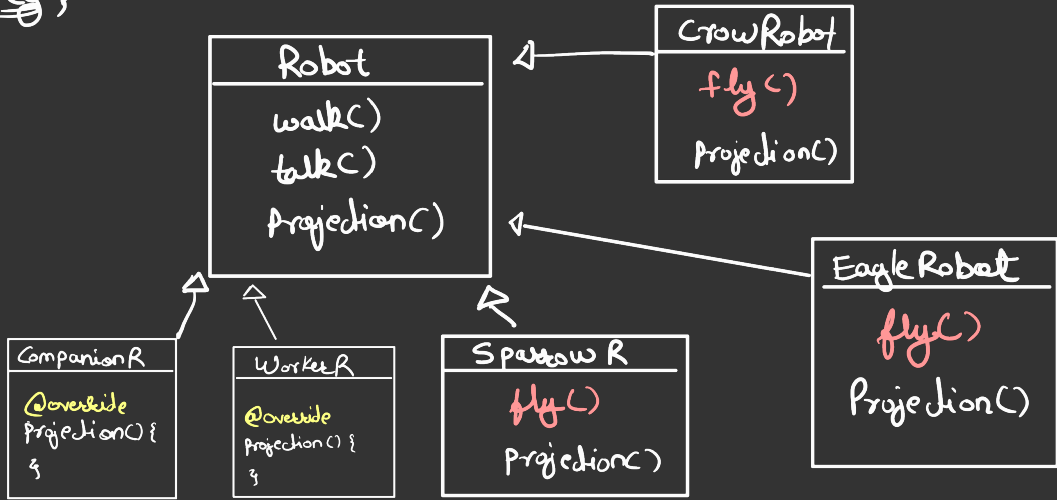


So far everything is good.

Now new types of robots are here, let's say **Sparrow Robot**



Ab future me multiple robots aaye jo fly kar sake hai
eg,

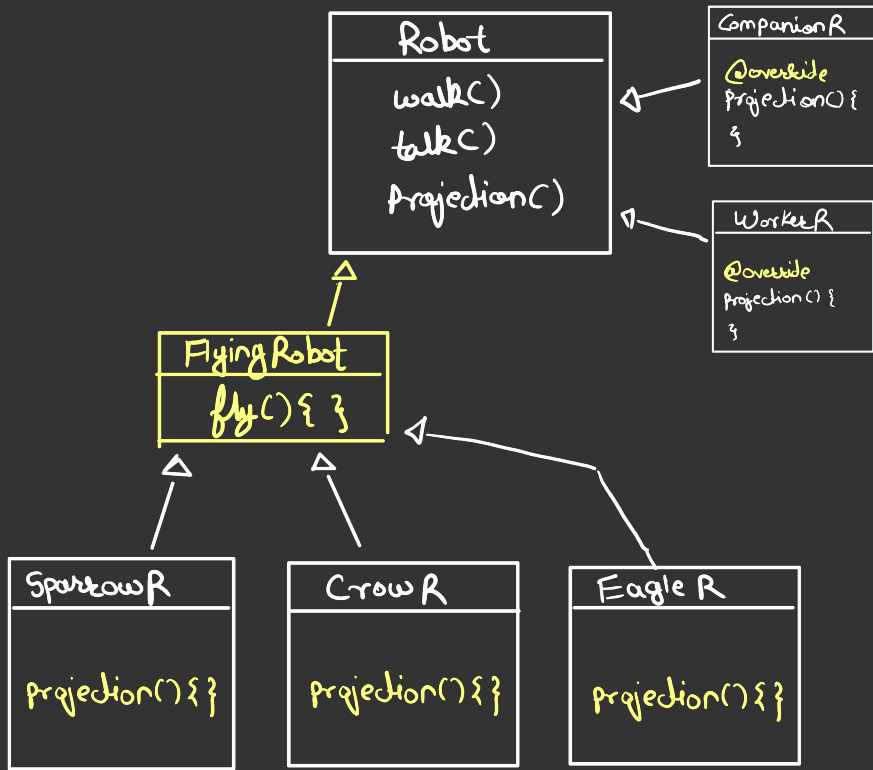


→ Ab ye jo fly method hai ye sabme same hai, redundant code !!

It is breaking very basic principal called
"DRY" [Donot repeat Yourself]

§ We can't move `fly()` method to Robot class as CompanionRobot & WorkerRobot should not fly.

→ To avoid this let's use one more level of inheritance

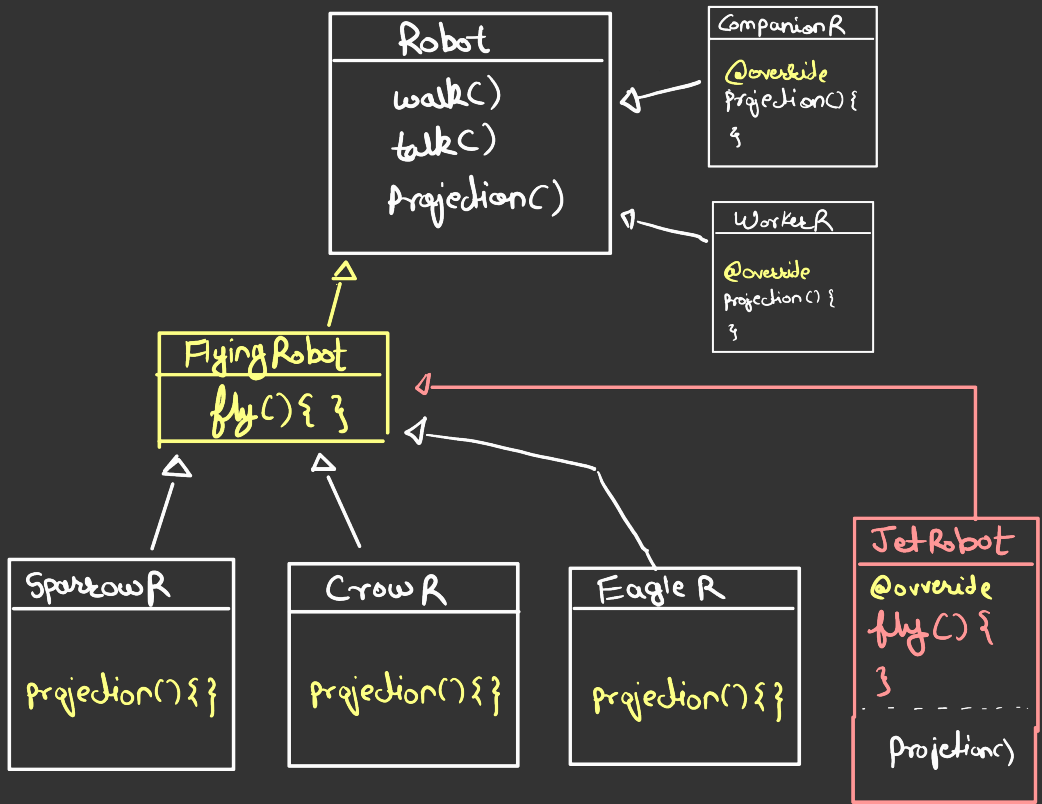


This case is solved but ye problem, bar bar aayegi,

Everytime we can't creat hierarchy !!

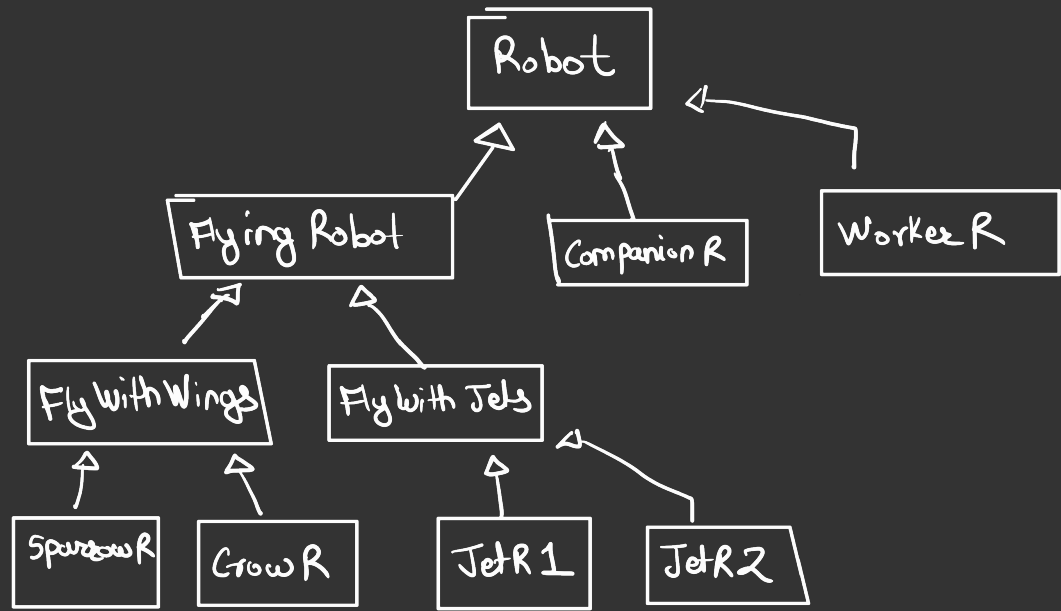
○ Kisi ek functionality se nayi hierarchy banana is not a correct way to deal this.

Now let's say, EK JetRobot aaya aur uski fly karne ka tarika bilkul different hai So it will override fly() method of FlyingRobot class



Now JetRobot jaise fly karne wale kuch aur robots aaye, ab unko bhi fly method samne haike se override karna hoga which is again Breaking "DRY" principle.

Now, Hierarchy first extend Karna padegi.



⊕ The solution to inheritance is not more Inheritance

⊕ Problems with Inheritance

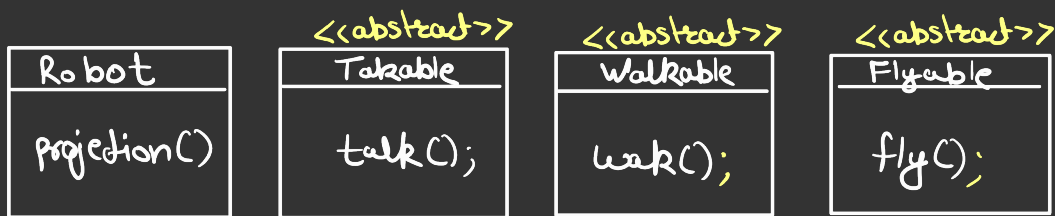
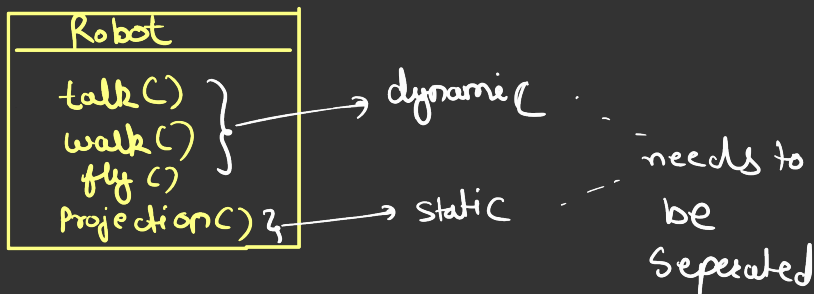
- ① Code Reuse
- ② To add new feature a lot of code changes were required.
- ③ Breaking OCP.

Solⁿ :→ Strategy Design Pattern

Strategy Design Pattern

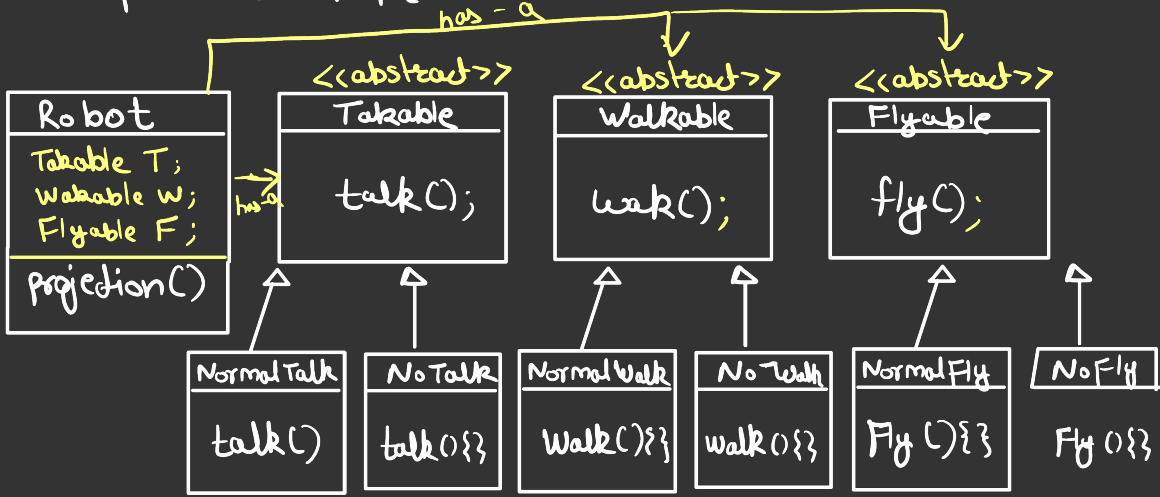
↳ Defines a family of Algorithms, put them into separate classes so that they can be changed at run time.

* Revisit Robot Usecase

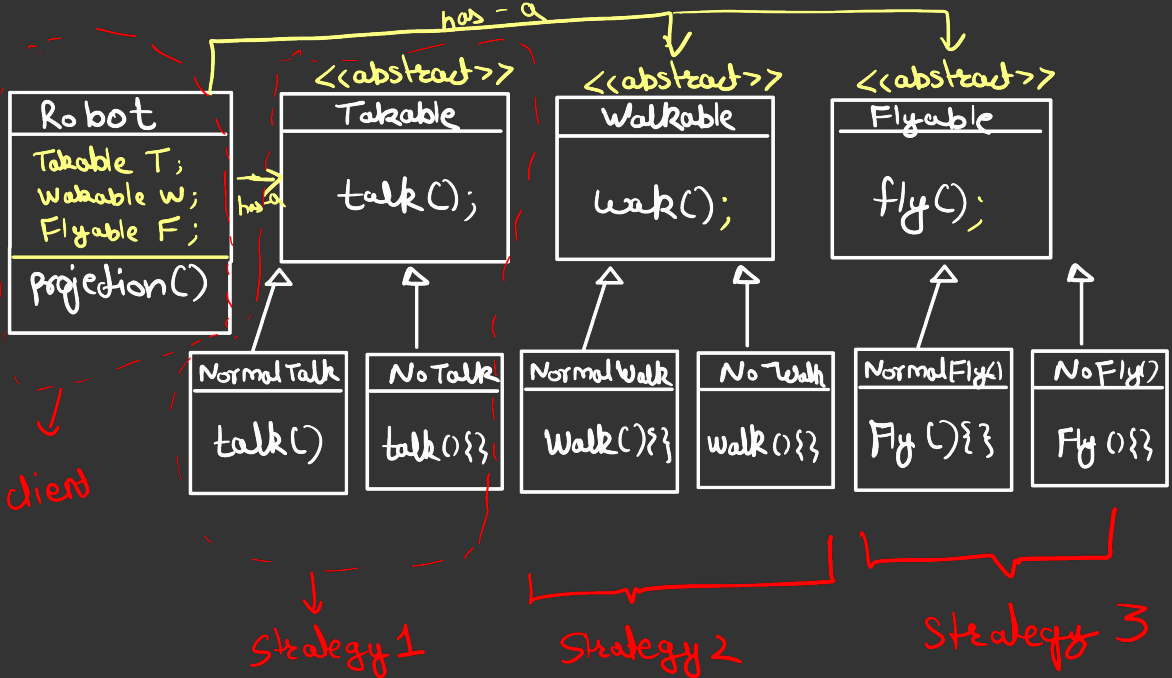


Interfaces

⊛ Now each Interface will have their concrete Implementation -



Sargon of Strategy Design Pattern :-



Now, Starting with Robot useCase with Strategy Pattern

Robot	
Tearable t ; Walkable w ; Flyably f ;	
walk()	- - -
talk()	- - -
fly()	- - -
projection();	

walk() {
 1 w.walk
}

talk() {
 1 t.talk()
}

simply delegate

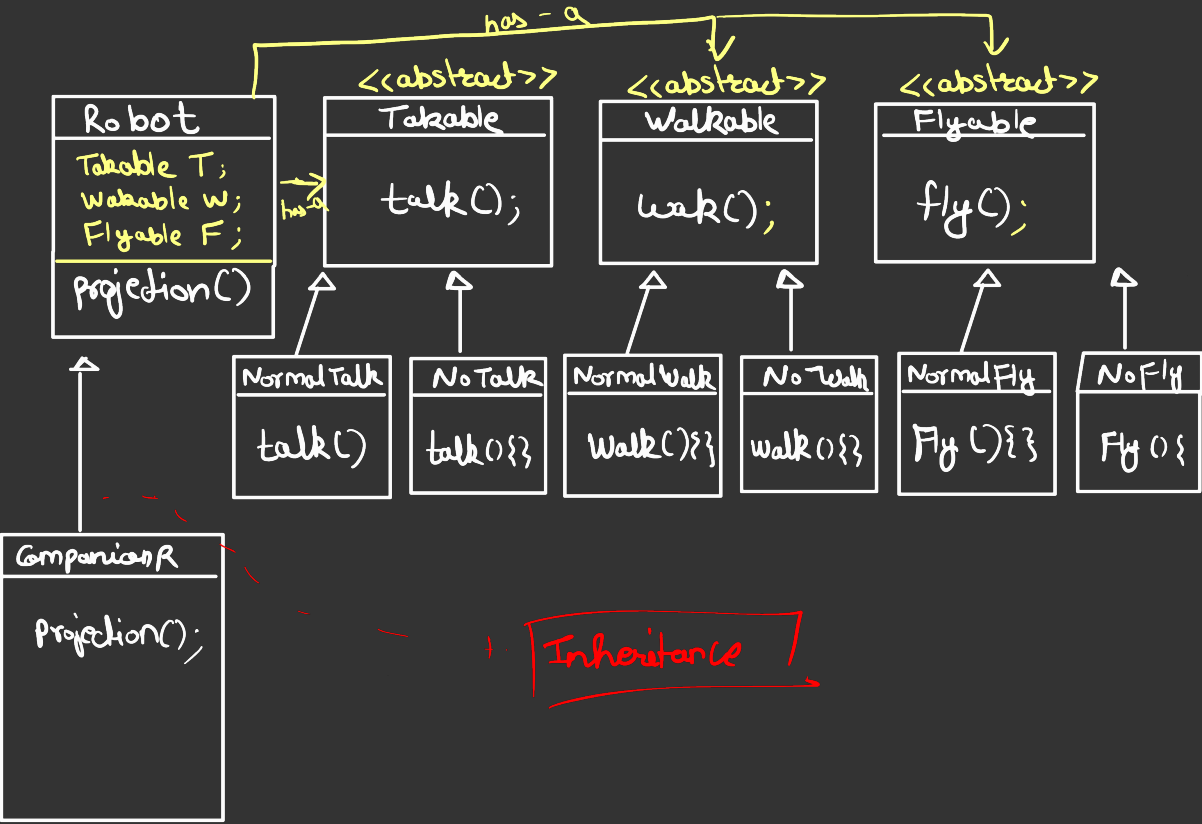
Let's Create a Companion Robot

↳ Robot robot = new CompanionRobot(
 new NormalTalk();
 new NormalWalk();
 new NoFly();
);

* CRUX of Strategy Design Pattern

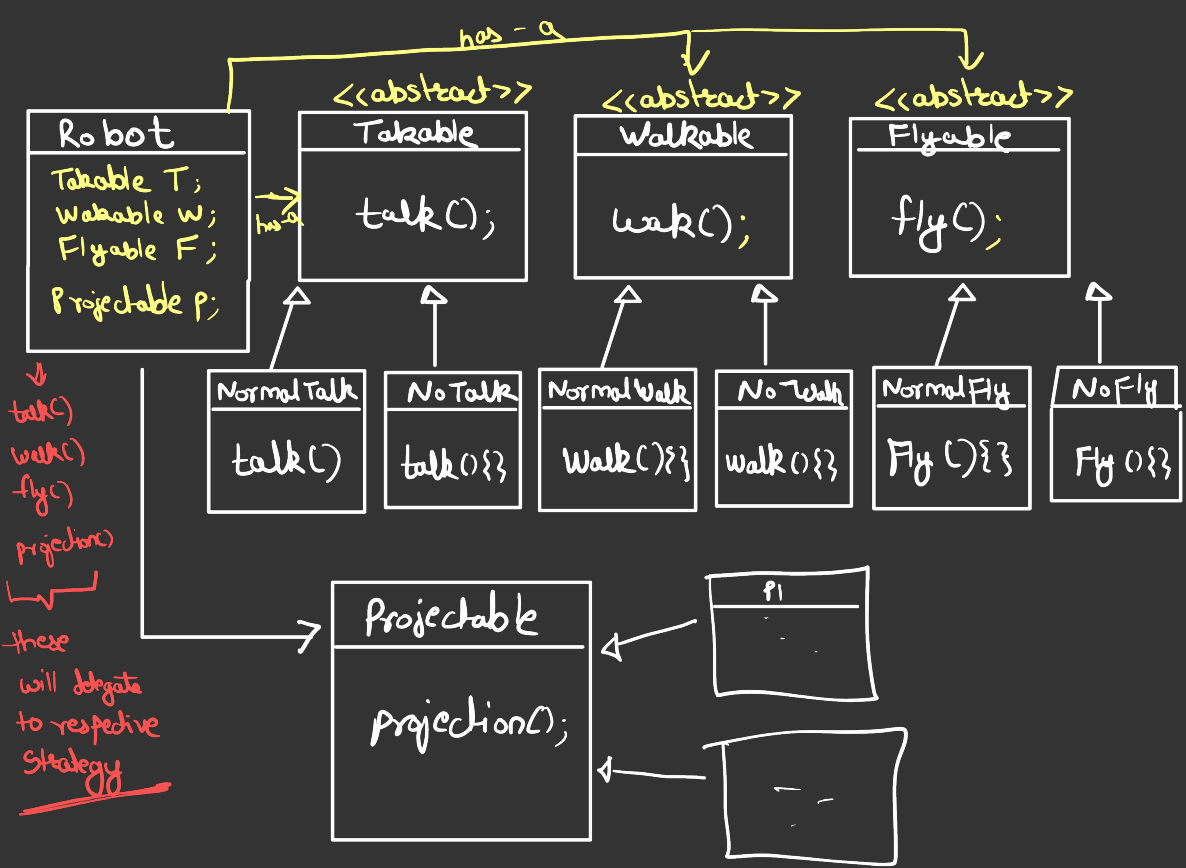
→ Favor Composition over Inheritance.

①



1. Inheritance

→ Agar hume inheritance pura tarah se samjhe karna hai to fir hume, projection() method ke liye bhi Composition use karna hoga Seperate Karke.

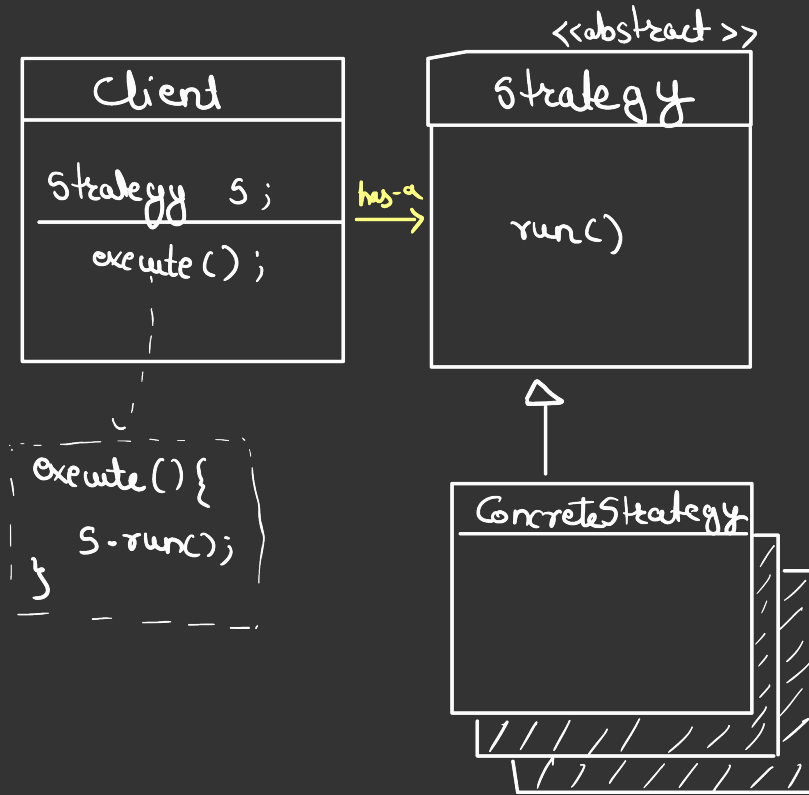


Now, no need of inheritance now,

```

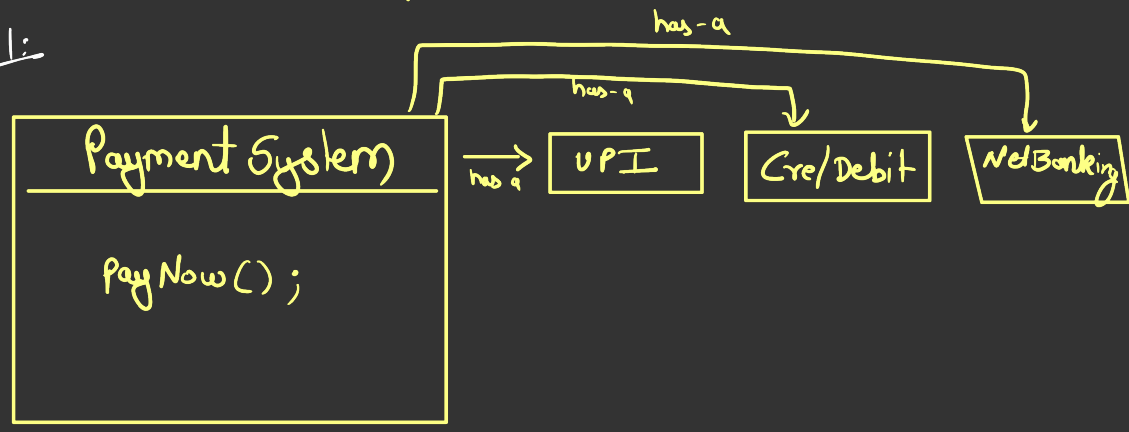
Robot robot = new CompanionR(
    new NormalTalk(),
    new NormalWalk(),
    new NoFly(),
    new P1()
)
  
```

* Standard UML diagram of Strategy Pattern



Real Time Example

eg 1:



eg 2:

